

Warfarin Dose Adjuster Assistant

AI-Powered Warfarin Dose Recommendation System for Mechanical Mitral Valve Patients

2. Target User, Workflow, and Business Value

Who the user is

The target user is an anticoagulation clinic nurse or clinical pharmacist who manages warfarin therapy for patients with mechanical mitral valve replacements. These clinicians review INR results and adjust warfarin doses at every patient visit, typically one to four times per month per patient.

What task is being improved

The task is weekly warfarin dose adjustment: given a patient's current INR, recent dose history, pharmacogenomic profile (CYP2C9/VKORC1 genotype), comorbidities, and concurrent medications, the clinician must decide whether to increase, maintain, decrease, or hold the dose. For mechanical mitral valve patients, the target INR is 2.5-3.5. Under-anticoagulation risks thromboembolic stroke; over-anticoagulation risks major bleeding.

Where the workflow begins and ends

- **Begins:** Patient presents with a new INR result from a laboratory draw or point-of-care device.
- **Ends:** The clinician documents the new weekly dose and schedules the next INR check.

Why this matters

Manual dose adjustment with paper-based nomograms is time-consuming, highly variable across clinicians, and does not systematically account for pharmacogenomics or drug interactions. This prototype explores whether a structured ML-based tool can support that decision with a faster, more consistent recommendation.

3. Problem Statement and GenAI Fit

The exact task the system performs

Given a structured patient record, INR, recent warfarin doses, CYP2C9/VKORC1 genotype, BMI, creatinine, concurrent medications, and dietary or illness flags, the system predicts the optimal weekly warfarin dose and returns a structured clinical recommendation including a confidence level, dose action, and tiered safety alerts.

What part of the workflow benefits from AI

Warfarin dose prediction is a high-dimensional regression problem. The right dose depends simultaneously on pharmacogenomics, longitudinal INR trends, drug interactions, and comorbidities, interactions that rule-based nomograms cannot capture. The system uses machine learning regression trained on a synthetic patient cohort and returns structured JSON output that is machine-readable and auditable rather than free-form text.

Why a simpler tool would not be enough

Standard nomograms use 3-5 variables, have no pharmacogenomic adjustment, no drug interaction layer, and no confidence calibration. This prototype addresses all three gaps: multi-variable regression, a 23-entry interaction lookup, and a tiered safety guardrail engine.

Prototype scope note: The system was built and tested on synthetic data only. It has not been validated against real patients or compared against a clinical baseline in a formal study.

4. Planned System Design and Baseline

What was built

The prototype consists of four components, all implemented and running locally.

Component 1: Synthetic patient dataset

- **Script:** generate_warfarin_db.py
- **Output:** warfarin_cohort_MMV.xlsx, 250 patients x 20 weeks = 5,000 rows
- **Features:** INR, weekly dose, CYP2C9/VKORC1 genotype, BMI, creatinine, comorbidity flags, drug/food interaction deltas, illness and adherence flags
- **Simulation:** INR modeled as a function of dose, genotypic sensitivity, and interaction effects with added Gaussian noise

Component 2: ML pipeline and model selection

- **Script:** warfarin_ml_pipeline.py
- **Split:** 80/20 train/test (3,800 training records, 1,000 test records)
- **Features engineered:** 33 tabular features including lag INR (1-3 weeks), lag dose, genotype encodings, and binary comorbidity flags
- **Results saved to:** model_comparison_results.csv

Model	RMSE (mg/wk)	R ²	% within ± 1 mg	Inference latency
Random Forest	0.261 mg	0.993	97.4%	~12 ms
XGBoost (selected)	0.128 mg	0.998	100%	~5 ms
LSTM	0.194 mg	0.996	98.8%	~150 ms

Component 3: FastAPI REST API

- **File:** warfarin_api/main.py
- **Server:** Uvicorn at http://127.0.0.1:8000, confirmed running; Swagger docs at /docs
- **Startup:** Model trains on launch, confirmed: "[Predictor] Ready. 3,800 training records."
- **5 endpoints:** GET /health, GET /model-info, POST /predict, POST /interaction-check, GET /patient/{patient_id}
- **Input validation:** Pydantic v2 schemas; invalid inputs return 422 with field-level errors

Component 4: Safety guardrail engine and interaction database

- **8 safety checks:** INR_CRITICAL_HIGH (>5.0), INR_CRITICALLY_LOW (<1.5), DOSE_UNUSUALLY_HIGH, DOSE_LARGE_CHANGE (>30%),

DRUG_INTERACTION_POTENTIATOR, DRUG_INTERACTION_REDUCER,
ADHERENCE_CONCERN, ILLNESS_INR_RISK

- **Severity tiers:** INFO / WARNING / CRITICAL
- **Interaction database:** 23 entries (14 drug, 9 food) in warfarin_api/interactions.py; exact and fuzzy (first-4-character) matching
- **Confidence scoring:** High / Moderate / Low based on count of active perturbing factors

Model and Provider Selection, Cost / Latency / Quality Trade-offs

Three architectures were evaluated on the same synthetic cohort. XGBoost was selected because it achieved the lowest RMSE (0.128 mg), the highest clinical tolerance (100% of predictions within +/-1 mg), and the fastest inference (~5 ms) without requiring GPU infrastructure. LSTM was eliminated on latency grounds, 150 ms is too slow for interactive clinical use. This applies the Week 2 framework of matching the model choice to quality, latency, and cost constraints simultaneously.

Tool Use / Function Calling

The 5 REST API endpoints function as callable tools with defined input schemas (Pydantic v2), typed JSON outputs, and scoped responsibilities. Each endpoint does one thing: /predict returns a dose recommendation, /interaction-check queries the pharmacological knowledge base, /patient/{id} retrieves longitudinal history. A downstream LLM agent could chain these tools in a ReAct loop without any changes to the API, applying the Week 4 tool call protocol directly.

Baseline

The intended comparison is against manual dose adjustment using the Warfarin Dosing Service (WDS) nomogram. That comparison has not been run yet. It is included as a planned next step, not a completed one. The current prototype establishes the AI side of that comparison only.

5. Evaluation Plan

What was actually tested

A 10-case automated test suite (warfarin_api/test_api.py) was run against the live API using the httpx library. All 10 tests passed.

1. **GET /health** — confirmed API is running and model is loaded
2. **GET /model-info** — confirmed training record count, feature count, and model type
3. **POST /predict (stable patient)** — INR 2.8, dose 30 mg/week; returned Maintain, High confidence, no alerts
4. **POST /predict (critical INR + antifungal)** — INR 5.8 + fluconazole; returned CRITICAL alert, Hold action
5. **POST /predict (poor metabolizer)** — CYP2C9 *3/*3 + VKORC1 AA; returned low dose, Moderate confidence
6. **POST /predict (enzyme inducer)** — rifampin; returned significant dose increase, drug interaction flag
7. **POST /interaction-check (fluconazole)** — confirmed HIGH severity potentiator entry returned
8. **POST /interaction-check (rifampin)** — confirmed HIGH severity reducer entry returned
9. **GET /patient/P042** — confirmed last 5 weeks of visit history returned

10. **422 validation error** — confirmed Pydantic rejects malformed input with field-level errors

ML model performance on held-out test set

The XGBoost model was evaluated on the 1,000-row held-out set (20% of the synthetic cohort). These metrics reflect performance against the synthetic data generator, not real clinical outcomes.

- RMSE: 0.128 mg/week
- R-squared: 0.998
- % predictions within +/-1 mg of ground truth: 100%
- % predictions within +/-2 mg of ground truth: 100%

All metrics are measured against synthetic ground truth only. The nomogram comparison has not been conducted.

6. Example Inputs and Failure Cases

Test cases run through the API

Case 1, Stable, in-range patient

- **Input:** INR 2.8, dose 30 mg/week, CYP2C9 *1/*1, VKORC1 GG, no interactions
- **Result:** Maintain, High confidence, no safety alerts

Case 2, Critical high INR with antifungal (fluconazole)

- **Input:** INR 5.8, dose 32.5 mg/week, concurrent fluconazole
- **Result:** Hold, CRITICAL alerts fired (INR_CRITICAL_HIGH + DRUG_INTERACTION_POTENTIATOR)

Case 3, Poor metabolizer

- **Input:** CYP2C9 *3/*3, VKORC1 AA (high genetic sensitivity)
- **Result:** Low dose recommendation, Moderate confidence, genotypic sensitivity flagged

Case 4, Enzyme inducer (rifampin)

- **Input:** INR 1.8, dose 35 mg/week, concurrent rifampin
- **Result:** Significant dose increase recommended, DRUG_INTERACTION_REDUCER flagged

Case 5, Invalid input (validation test)

- **Input:** Malformed request body missing required fields
- **Result:** 422 Unprocessable Entity returned with field-level Pydantic error detail

Anticipated failure cases

Failure 1, Multiple simultaneous drug interactions

The engine applies drug effects additively. A patient on both fluconazole and amiodarone may have a combined INR impact larger than the sum of individual effects. The prototype may underestimate this and miss a CRITICAL threshold.

Failure 2, Drug not in the 23-entry knowledge base

Any medication not catalogued in interactions.py returns no interaction flag, a silent false negative. The prototype currently has no mechanism to warn the clinician when a submitted drug name is unrecognized.

Failure 3, Synthetic data distribution vs. real patients

The model was trained on data generated from pharmacological equations. Real patient INR trajectories include noise from adherence variability, dietary inconsistency, and measurement error that the synthetic generator does not model. Prototype performance on real data is unknown.

7. Risks and Governance

Where the system could fail

- **Synthetic training data:** The model has never seen real patient data. Its confidence on real-world inputs is unknown and likely overstated.
- **Interaction database coverage:** Only 23 drug/food entries. Any agent not in the list passes through without a flag.
- **No real baseline comparison:** The prototype cannot yet demonstrate improvement over manual dose adjustment.
- **Input quality:** Data entry errors propagate directly into recommendations with no detection.

Where the system should not be trusted

- Any clinical decision with a real patient, this is a first prototype on synthetic data only
- Pediatric patients (model trained on adult pharmacokinetics)
- Pregnant patients (warfarin is contraindicated in pregnancy)
- Patients with mechanical aortic valves (different INR target: 2.0-3.0)
- Patients with severe hepatic or renal impairment
- Any drug not in the 23-entry knowledge base

Controls in the current prototype

- **CRITICAL alert classification:** INR > 5.0 and INR < 1.5 are flagged as CRITICAL in the API response.
- **Dose-change threshold flag:** Recommendations that change the prior dose by more than 30% trigger DOSE_LARGE_CHANGE in the alert list.
- **No write access:** The API is read-only and runs locally. It does not connect to any EHR or patient record system.

Data and privacy

The prototype uses only synthetic data. No real patient health information (PHI) was used at any stage. In a future real-world deployment, all patient inputs would constitute protected health information and require HIPAA-compliant infrastructure.

8. Plan for the Week 6 Check-in

What is running now (as of today)

- **warfarin_cohort_MMV.xlsx** — 5,000-row synthetic cohort generated and saved to the project folder
- **warfarin_ml_pipeline.py** — RF / XGBoost / LSTM compared; results saved to model_comparison_results.csv
- **warfarin_api/main.py** — FastAPI, all 5 endpoints live, Swagger docs at /docs
- **warfarin_api/interactions.py** — 23-entry drug/food interaction database with fuzzy matching
- **warfarin_api/test_api.py** — 10-case automated test suite; all tests passed

What evaluation is in place

- ML performance metrics on 1,000-row held-out set: RMSE 0.128 mg, R2 0.998, 100% within +/-1 mg
- 10 live API test cases covering stable patient, critical INR + drug interaction, poor metabolizer, enzyme inducer, invalid input
- Safety alert firing verified for CRITICAL scenarios (fluconazole + high INR, rifampin)
- Input validation verified via 422 error test

What is still needed

- Formal baseline comparison against WDS nomogram on matched patient scenarios, not yet run
- Unrecognized drug name warning in /predict response
- Explicit prototype disclaimer in all /predict responses
- Expanded test set covering edge cases: polypharmacy, renal impairment, missed doses